



Artificial Intelligence Software, Lecture 9

Deep Learning Deployment

Triton Inference Server and TensorRT Optimization



Center for
Cognitive
Modeling



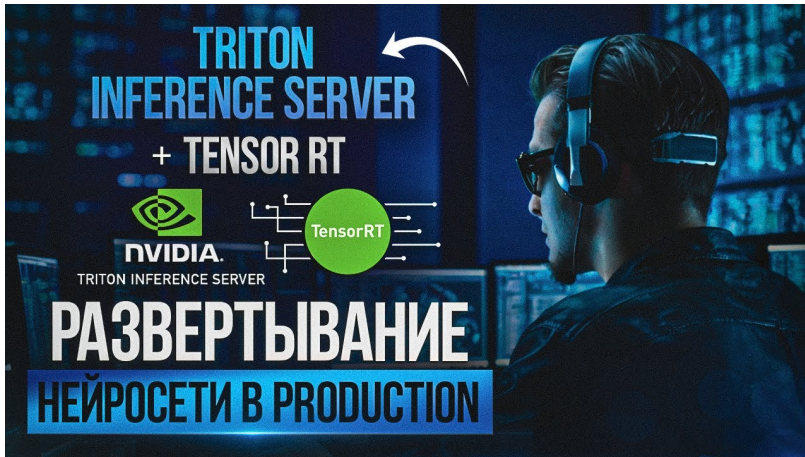
Lecture Plan

1. Why production inference differs from research
2. **Model Export:** PyTorch → ONNX
3. **Triton Inference Server:** structure, config, Docker
4. **FastAPI** wrapper + **tritonclient** (gRPC)
5. **TensorRT** optimization: FP16, trtexec
6. **Monitoring:** Prometheus + Grafana
7. **Advanced Triton:** Dynamic Batching, Instance Groups
8. **Load Testing:** Locust



Tutorial

Video





References

- Методы сжатия нейронных сетей (дистилляция, квантизация, pruning)
- Мониторинг и Логи ПРОДАКШЕН уровня — Grafana + Loki + Prometheus + Promtail
- Comparing AI Inference Optimization Tools: TensorRT vs ONNX Runtime vs Triton
- Server Monitoring // Prometheus and Grafana Tutorial



Research vs. Production Inference

Why can't we just `model.predict()`?

Research / Notebook

- One request at a time
- No latency SLA
- GPU underutilized between calls
- Framework-specific code everywhere

Production Server

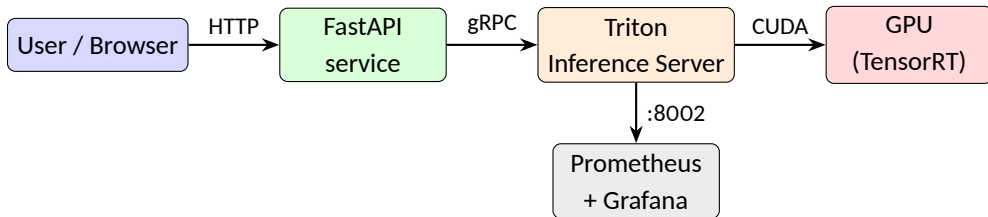
- Hundreds of concurrent users
- P99 latency < 100 ms
- GPU must stay busy
- Hardware-optimized execution graph

A production inference stack decouples the model from the serving logic and optimizes each layer independently.



The Full Stack We Will Build

End-to-end deployment architecture



Ports: 8000 HTTP • 8001 gRPC • 8002 metrics (Prometheus)



What Is ONNX?

Open Neural Network Exchange

ONNX is a hardware-agnostic, open format for storing trained neural network graphs.

Feature	Meaning
Portable	Run the same .onnx file with PyTorch, TensorRT, OpenVINO, CoreML...
Graph-based	Operators (Conv, ReLU, BN) are stored as a static computation graph
Runtime-agnostic	No Python, no PyTorch dependency on the server
Optimizable	Graph-level optimizations before hardware compilation

ONNX is the *lingua franca* of deep learning inference. Export once — deploy anywhere.



Exporting ResNet-152 to ONNX

1. Load model and append Softmax

```
backbone = models.resnet152(weights="DEFAULT")
```

```
backbone.eval()
```

```
model = torch.nn.Sequential(backbone, torch.nn.Softmax(dim=1))
```

2. Dummy input (any valid shape; used for tracing)

```
dummy = torch.randn(1, 3, 224, 224)
```

3. Export

```
torch.onnx.export(
```

```
    model, dummy, "resnet152.onnx",
```

```
    input_names=["input"],
```

```
    output_names=["output"],
```

```
    dynamic_axes={                # allow variable batch size
```

```
        "input": {0: "batch_size"},
```

```
        "output": {0: "batch_size"}},
```

```
    opset_version=17,)
```




Why Dynamic Axes?

Making the model batch-size agnostic

Without `dynamic_axes` the ONNX graph has a *fixed* batch dimension of 1.

- Triton and TensorRT need to send batches larger than 1 to saturate the GPU.
- A fixed-shape model will reject any input tensor whose batch size $\neq 1$.
- Dynamic axes mark axis 0 of input and output as symbolic ("batch_size"), so the runtime can reshape them freely.

Always set `dynamic_axes` for at least the batch dimension when targeting a production server.



Why Add Softmax?

Convenience modification before export

- ResNet's last layer produces raw **logits** — un-normalized scores.
- Downstream code then needs to apply softmax itself every time.
- By appending `torch.nn.Softmax(dim=1)` *before* export, the ONNX model outputs class **probabilities** directly.
- The FastAPI response can read `argmax` straight from the Triton output.

Rule of thumb: bake post-processing into the graph whenever the extra operation is cheap and always applied.



NVIDIA Triton Inference Server

What it is and what it solves

Triton is a production-grade, open-source inference serving system by NVIDIA.

Capability	Details
Multi-framework	ONNX Runtime, TensorRT, TensorFlow, PyTorch
Dynamic Batching	Aggregates concurrent single requests into one GPU batch
Instance Groups	Runs N copies of a model in parallel on one or more GPUs
HTTP + gRPC	Clients can use either protocol
Metrics	Exposes latency, throughput, GPU utilization



Triton: Model Repository Structure

The required folder layout

Triton expects a strict directory hierarchy:

```
models/  
  resnet152/          # model name (used in API calls)  
    config.pbtxt      # model configuration file  
    1/                # version number (integer folder)  
      model.onnx       # the model file
```

Key rules:

- The version folder *must* be a number (1, 2, ...).
- The file inside must be named model.onnx (or model.plan for TensorRT).
- Multiple versions can coexist; Triton serves the latest by default.



FastAPI Wrapper: Architecture

Why an extra API layer?

Triton speaks tensors.

It expects raw FP32 arrays with exact shapes.

The user sends a .jpg file. Someone must bridge the gap.

FastAPI service responsibilities:

1. Accept multipart image upload
2. Run preprocessing pipeline
3. Call Triton via tritonclient
4. Return JSON with top class

The FastAPI layer is *thin*: no model logic, no weights. Preprocessing + glue code only.



FastAPI + tritonclient: gRPC Inference

```
async def triton_infer(image_np: np.ndarray) -> np.ndarray:
    client = grpcclient.InferenceServerClient("localhost:8001")
    inp = grpcclient.InferInput("input", image_np.shape, "FP32")
    inp.set_data_from_numpy(image_np)
    out = grpcclient.InferRequestedOutput("output")
    response = await client.infer(
        model_name="resnet152",
        inputs=[inp],
        outputs=[out],)
    return response.as_numpy("output")    # shape: (1, 1000)
```

gRPC vs HTTP: gRPC serialization is $\sim 2\times$ faster for large tensors; use it whenever the client supports it.



TensorRT: What and Why

Hardware-specific compilation of the inference graph

TensorRT compiles an ONNX model into a hardware-optimized .plan engine.

Optimization	Effect
Layer fusion	Merges Conv+BN+ReLU into a single CUDA kernel
Kernel autotuning	Selects the fastest GEMM algorithm for the exact GPU
FP16 / INT8	Halves or quarters memory bandwidth; faster tensor core ops
Memory pooling	Pre-allocates GPU buffers, removes allocation overhead

A TensorRT-compiled ResNet-152 in FP16 runs **2-4× faster** than ONNX Runtime on the same GPU.



Serving TensorRT with Triton

Switching from ONNX to TensorRT engine --- only two changes

1. Place model.plan in the version folder instead of model.onnx.
2. Update config.pbtxt:

ONNX Runtime

platform: "onnxruntime_onnx"

TensorRT

platform: "tensorrt_plan"

Everything else (FastAPI, tritonclient calls, monitoring) stays identical.

The engine file is GPU-specific. Recompile with trtexec when moving to a different GPU model.



Monitoring: Prometheus + Grafana

Observability for inference servers

Triton exposes a **Prometheus-compatible** metrics endpoint at :8002/metrics.

Metric	Meaning
nv_inference_request_success	Total successful inference requests
nv_inference_queue_duration_us	Time requests spend waiting in queue
nv_inference_compute_infer_duration_us	Pure GPU compute time
nv_gpu_utilization	GPU utilization %
nv_gpu_memory_used_bytes	GPU memory consumed

Computed from raw metrics: RPS, P50/P95/P99 latency, avg batch size — derive these in Grafana via PromQL.



Dynamic Batching

Triton's killer feature for high throughput

Without dynamic batching:

- Each HTTP request $\Rightarrow$ 1 GPU forward pass
- GPU is mostly idle between requests
- Low throughput under concurrent load

With dynamic batching:

- Triton queues requests for up to $N \mu s$
- Merges them into a single large batch
- 1 GPU forward pass serves many clients

Add to config.pbtxt:

```
dynamic_batching {  
    max_queue_delay_microseconds:  
        100  
}
```

Triton will merge up to `max_batch_size` requests within the delay window.



Instance Groups

Running multiple model copies in parallel

Even with dynamic batching, a single model instance can become a bottleneck at very high RPS.

Instance Groups let Triton load N copies of the model on the GPU simultaneously, each serving requests independently.

For small models (MobileNet-level), count: 2--4 often doubles throughput. For large models (ViT-Large), VRAM is the limit.



Load Testing with Locust

Simulating real user traffic

Locust is a Python load-testing tool that spawns many virtual users, each running your task functions asynchronously.

- Define user behavior in pure Python — no XML, no DSL.
- Web UI shows RPS, response times, and failures in real time.
- Ramp users up gradually to find the saturation point.
- Use together with Grafana to see GPU utilization vs. RPS.



Summary: The Complete Pipeline

What you will implement in the tutorial

1. **Export:** `torch.onnx.export` with `dynamic_axes` + Softmax head
2. **Deploy:** Triton in Docker, `config.pbtxt`, ONNX Runtime backend
3. **Wrap:** FastAPI + `tritonclient.gRPC`, TorchVision preprocessing
4. **Optimize:** `trtexec --fp16`, switch to `tensorrt_plan` platform
5. **Monitor:** Prometheus scrapes `:8002`, Grafana dashboard for latency & GPU
6. **Tune:** Dynamic batching delay + instance group count
7. **Verify:** Locust load test, compare RPS before/after each optimization

Deserve "A" grade!

– Oleg Bulichev

✉ bulichev.ov@mipt.ru

📍 @Lupasic

🏠 Цифра, 521